

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Ben Youcef Benkhedda University – Algiers 1
Faculty of Sciences
Department of Computer Science

Bachelor's Thesis

in Computer Science

Specialty: Engineering of Information Systems and Software

**Development of a Web-Based ERP
System
for Pharmaceutical Laboratory
Management**

Case Study: Inphamedis

Supervised by:

– Mrs. Zairi Hadjer

Presented by:

– Benamara M'hamed Anis

– Malek Ahmed Amir

2024/2025

Contents

1	Project Framework	2
	Introduction	2
1.1	Presentation of the Case Study Organization	2
1.1.1	Introduction	2
1.1.2	Presentation of Inphamedis	3
1.1.3	Organizational Structure	3
1.2	Existing Context	4
1.3	Problem Statement	5
1.4	Project Objectives	5
1.4.1	General Objective	5
1.4.2	Specific Objectives	5
1.4.3	Expected Results	6
1.5	Proposed Solution	6
	Conclusion	8
2	Analysis and Conception	9
2.1	Introduction	9
2.2	Specification of Needs	9
2.2.1	General Description of the Solution	9
2.2.2	Capture of Functional Needs	10
2.3	The Conception (Design)	13
2.3.1	Presentation of UML	13
2.3.2	Use Case Diagrams	13
2.3.3	Sequence Diagrams	17
2.3.4	Class Diagram	21
2.3.5	Security Considerations	23
	Conclusion	23

Chapter 1

Project Framework

Introduction

The pharmaceutical industry operates in one of the most demanding and regulated environments in the world. It requires precise and coordinated management of all operational processes, from the procurement of raw materials to the distribution of finished medicinal products, while strictly adhering to quality standards and regulatory requirements such as Good Manufacturing Practices (GMP).

In this context, information systems have become a critical pillar in ensuring traceability, operational efficiency, and regulatory compliance within pharmaceutical organizations. The absence of an integrated management solution exposes laboratories to significant risks, including data loss, lack of traceability, and non-compliance with pharmaceutical regulations.

This chapter presents the general framework of our project. We begin by introducing the case study organization, Inphamedis, and describing its core activities. We then analyze the existing management context and identify its main limitations. Based on this analysis, we formulate the problem statement and define the project objectives before presenting the proposed solution.

1.1 Presentation of the Case Study Organization

1.1.1 Introduction

In this section, we present Inphamedis, the pharmaceutical laboratory that serves as the case study for this project. We provide an overview of its mission, core activities, and organizational structure in order to understand the operational context in which

the proposed system will be deployed.

1.1.2 Presentation of Inphamedis

Inphamedis is an Algerian pharmaceutical laboratory specialized in the production, quality control, and distribution of medicinal products. As a company operating in a highly regulated sector, Inphamedis is required to comply with national and international pharmaceutical standards, including Good Manufacturing Practices (GMP), in order to guarantee the safety, efficacy, and quality of its products.

The laboratory's core activities include:

- **Pharmaceutical Production:** Manufacturing of medicinal products in controlled environments following strict batch-based fabrication procedures, ensuring product consistency and compliance with quality standards.
- **Stock and Inventory Management:** Handling of raw materials, packaging materials, and finished goods with rigorous control over quantities, storage conditions, and expiry dates.
- **Procurement and Supply Chain:** Managing relationships with certified suppliers and ensuring the timely and compliant acquisition of raw materials necessary for production.
- **Quality Control:** Conducting systematic laboratory analysis and testing of each production batch before it is approved for release and distribution.
- **Sales and Distribution:** Managing client orders and ensuring the safe, traceable, and timely delivery of pharmaceutical products to clients and distributors.

1.1.3 Organizational Structure

Inphamedis is organized around five main operational departments, each responsible for a critical function within the pharmaceutical supply chain:

- **Production Department:** Responsible for manufacturing pharmaceutical products according to approved formulas and production orders.
- **Warehouse and Stock Department:** In charge of receiving, storing, and tracking raw materials and finished products.
- **Procurement Department:** Manages supplier relationships, purchase requests, and the acquisition of materials needed for production.
- **Quality Control Department:** Performs analysis and testing of production batches and ensures compliance with quality standards.

- **Sales and Distribution Department:** Handles client orders, delivery planning, and invoicing.

These departments must operate in close coordination to ensure the smooth and compliant flow of operations across the entire pharmaceutical value chain.

1.2 Existing Context

In the pharmaceutical sector, the management of operational processes requires a high level of rigor and precision. At Inphamedis, the five operational departments — production, stock, procurement, quality control, and sales — must collaborate continuously to ensure the proper functioning of the laboratory.

Currently, the management of these operations relies mainly on traditional and non-integrated methods, including:

- The use of paper documents to record production orders and quality control results
- Excel spreadsheets for inventory tracking and stock management
- Email exchanges for communication between departments
- Manual archiving of batch records and intervention history
- Isolated software tools used independently by each department

Although these methods provide a basic level of organization, they present significant shortcomings in an environment as demanding as pharmaceutical manufacturing:

- Lack of data centralization and real-time visibility across departments
- Difficulty in quickly accessing and sharing operational information
- High risk of human error due to manual data entry and processing
- Absence of end-to-end batch traceability from raw material to final delivery
- Inability to automatically monitor expiry dates and stock minimum thresholds
- Loss or deterioration of critical documents such as batch records and quality reports
- Difficulty in planning and coordinating production and procurement operations
- Absence of automated performance indicators to support decision-making

In a regulated environment such as pharmaceutical manufacturing, these limitations can have serious consequences, including regulatory non-compliance, quality incidents,

product recalls that cannot be managed efficiently, and potential risks to patient safety. The current system therefore does not meet the requirements of a modern, integrated pharmaceutical management solution.

1.3 Problem Statement

In an operational context where quality, traceability, and regulatory compliance are non-negotiable priorities, the absence of an integrated management system represents a critical challenge for Inphamedis.

The main problem of this project can be formulated as follows:

How can we design and implement a centralized, integrated, and reliable web-based ERP system that ensures efficient management of all pharmaceutical laboratory operations — from raw material procurement to product distribution — while guaranteeing end-to-end batch traceability and regulatory compliance?

It therefore becomes necessary to develop a web application adapted to the specific operational needs of a pharmaceutical laboratory, capable of integrating all core management modules within a unified and coherent platform.

1.4 Project Objectives

1.4.1 General Objective

The main objective of this project is to design and develop a web-based ERP system for pharmaceutical laboratory management, tailored to the specific needs of Inphamedis. This system aims to centralize data, integrate all operational modules, and ensure complete traceability of pharmaceutical products throughout their entire lifecycle.

1.4.2 Specific Objectives

The project aims to:

- Design and implement a Stock Management module enabling real-time inventory monitoring, automated low-stock alerts, and expiry date management.
- Design and implement a Production Management module enabling the creation, planning, and tracking of manufacturing orders with automated batch number generation.

- Design and implement a Purchase Management module enabling the management of purchase requests, supplier selection, order tracking, and delivery reception.
- Design and implement a Quality Control module enabling the creation of control files, analysis result entry, batch validation or rejection, and analysis certificate generation.
- Design and implement a Sales Management module enabling client order management, stock availability verification, delivery note generation, and invoicing.
- Implement an end-to-end batch traceability system linking each batch to all its associated operations across all modules.
- Secure data access through a role-based authentication and authorization system.
- Provide real-time dashboards and performance indicators to support management decision-making.

1.4.3 Expected Results

Through this application, we aim to achieve:

- Full integration and centralization of all pharmaceutical laboratory management operations within a single unified platform.
- Complete and automated traceability of each product batch from raw material reception to client delivery, enabling rapid and targeted product recalls when necessary.
- Significant reduction of human errors associated with manual data entry and paper-based processes.
- Improved operational efficiency through automated alerts, real-time stock monitoring, and coordinated production planning.
- Strengthened regulatory compliance through systematic quality control recording and complete intervention history.
- Better decision-making supported by real-time performance indicators and consolidated management reports.

1.5 Proposed Solution

To address the identified limitations of the current management system at Inphamedis, we propose the development of a web-based ERP system specifically designed for pharmaceutical laboratory management.

The proposed solution consists of implementing a centralized digital platform that integrates five core functional modules:

- **Stock Management:** Real-time monitoring of inventory levels, automated low-stock alerts, expiry date management, storage location tracking, and batch blocking capabilities.
- **Production Management:** Creation and planning of manufacturing orders, resource assignment, production progress tracking, raw material consumption recording, and automated batch number generation.
- **Purchase Management:** Management of purchase requests, supplier selection and evaluation, purchase order generation, delivery reception, and invoice recording.
- **Quality Control:** Creation of control files, laboratory analysis result entry, batch validation or rejection, non-conformity management, batch recall triggering, and analysis certificate generation.
- **Sales Management:** Client order management, stock availability verification, delivery note generation, client invoicing, delivery tracking, and product return management.

The key differentiating feature of the proposed system is its **end-to-end batch traceability mechanism**. Every batch is assigned a unique identifier at the moment of production, and this identifier is systematically linked to all subsequent operations across all modules — stock movements, quality control records, purchase receptions, sales orders, and delivery notes. This mechanism allows the organization to:

- Instantly retrieve the complete history of any batch, from its raw material origin to its final delivery destination.
- Trigger targeted and efficient product recalls with full visibility of affected clients.
- Generate complete traceability reports to demonstrate regulatory compliance.
- Detect and respond to quality incidents in real time.

The system will be developed as a modern web application using a JavaScript-based technology stack. The backend will be built with **Node.js** and **Express.js**, providing a scalable and performant server-side environment, while the frontend will deliver a responsive and intuitive user interface. The application will include a secure authentication system with role-based access control adapted to the different user profiles of the laboratory.

Conclusion

In this chapter, we presented the general framework of our project. We introduced Inphamedis as a pharmaceutical laboratory operating in a highly regulated environment, and analyzed the significant limitations of its current manual and non-integrated management approach. This analysis allowed us to formulate a clear problem statement and define the objectives of the proposed solution.

To address the identified challenges, we proposed the development of a web-based ERP system integrating five core operational modules, with end-to-end batch traceability as a central and differentiating feature. This solution aims to centralize data, improve operational efficiency, and ensure full regulatory compliance across all laboratory activities.

The following chapter will focus on the analysis and design of the proposed system, presenting the UML models that formalize its functional and structural aspects.

Chapter 2

Analysis and Conception

2.1 Introduction

The development of a reliable and coherent software system requires a rigorous analysis and design phase before any implementation can begin. This phase transforms the identified user needs into a structured and formalized model that guides the development process and ensures consistency between functional requirements and technical solutions.

In this chapter, we present the analysis and conception of the proposed web-based ERP system for Inphamedis. We begin with a specification of needs, covering both a general description of the solution and a capture of its functional requirements. We then move to the conception phase, where the system is formalized through a set of UML diagrams that describe its functional, dynamic, and structural aspects.

To model the system, we adopt the Unified Modeling Language (UML) as a standard modeling framework, using use case diagrams, sequence diagrams, and a class diagram.

2.2 Specification of Needs

2.2.1 General Description of the Solution

The proposed solution is a web-based ERP system designed specifically for the management of pharmaceutical laboratory operations. It aims to replace the current fragmented and manual approach used at Inphamedis with a centralized, integrated, and automated platform that covers all core operational processes.

The system is built around five functional modules — Stock Management, Production Management, Purchase Management, Quality Control, and Sales Management — all

connected through a unified data layer. The central differentiating feature of the system is its end-to-end batch traceability mechanism, which links every product batch to all operations it undergoes across all modules, from raw material reception to final client delivery.

The application follows a client-server architecture:

- The **frontend**, developed with React.js, provides a responsive and intuitive user interface accessible via a web browser.
- The **backend**, built with Node.js and Express.js, handles business logic, data processing, and RESTful API services.
- The **database**, powered by PostgreSQL, ensures reliable and structured storage of all system data.

Access to the system is secured through a role-based authentication and authorization mechanism, ensuring that each user only accesses the functionalities relevant to their role within the laboratory.

2.2.2 Capture of Functional Needs

Identification of Actors

The system involves the following actors:

- **Administrator:** Manages user accounts, system configuration, and access control. Has full visibility over all modules.
- **Warehouse Keeper:** Responsible for recording stock movements and managing storage locations on a day-to-day basis.
- **Stock Manager:** Supervises inventory levels, generates stock reports, and manages expiry dates and batch blocking.
- **Production Manager:** Creates and plans manufacturing orders, assigns resources, and oversees production progress.
- **Production Operator:** Executes manufacturing operations and records raw material consumption.
- **Purchase Manager:** Manages purchase requests, supplier selection, purchase orders, and supplier evaluation.
- **Quality Manager:** Creates control files, validates or rejects batches, manages non-conformities, and triggers batch recalls when necessary.

- **Lab Technician:** Performs laboratory analyses and enters analysis results into the system.
- **Sales Manager:** Manages client orders, generates delivery notes, invoices clients, and handles product returns.
- **Client:** External actor who consults the product catalog and tracks the status of their deliveries.

Functional Requirements

The system must fulfill the following functional requirements, organized by module:

Stock Management:

- Allow users to view real-time stock levels for all products and raw materials.
- Record and track all stock movements (entries and exits) linked to batch numbers.
- Manage storage locations within the warehouse.
- Automatically generate alerts when stock falls below defined minimum thresholds.
- Monitor and manage product expiry dates.
- Enable tracking of any batch by its unique identifier.
- Allow the quality manager to block non-compliant batches.
- Generate periodic stock reports.

Production Management:

- Allow the creation and planning of manufacturing orders.
- Track production progress in real time.
- Record raw material consumption per manufacturing order.
- Assign human and material resources to production orders.
- Automatically generate a unique batch number upon order creation.
- Allow closure of manufacturing orders after production validation.

Purchase Management:

- Allow the creation of purchase requests for needed materials.
- Enable supplier selection and purchase order generation.
- Track the status of purchase orders (sent, in progress, received).

- Register deliveries and link them to batch numbers for traceability.
- Record supplier invoices and evaluate supplier performance.

Quality Control:

- Allow the creation of quality control files for each batch.
- Enable laboratory technicians to enter analysis results.
- Allow the quality manager to validate or reject batches based on results.
- Manage non-conformities and define corrective actions.
- Trigger targeted batch recalls in case of serious non-conformity.
- Generate official analysis certificates for validated batches.
- Automatically alert the quality manager when an anomaly is detected.

Sales Management:

- Allow the creation and management of client sales orders.
- Automatically verify stock availability upon order creation.
- Generate delivery notes linked to specific batch numbers.
- Invoice clients and track delivery status.
- Manage product returns with full batch identification.

Non-Functional Requirements

In addition to functional requirements, the system must satisfy the following non-functional requirements:

- **Security:** All data must be protected through secure authentication (JWT tokens) and role-based access control.
- **Performance:** The system must handle concurrent users and respond to requests within acceptable time limits.
- **Scalability:** The modular architecture must allow future addition of new modules without affecting existing ones.
- **Usability:** The user interface must be intuitive, responsive, and accessible from standard web browsers.
- **Reliability:** The system must ensure data integrity and availability during normal operating conditions.

2.3 The Conception (Design)

2.3.1 Presentation of UML

Definition of UML

The Unified Modeling Language (UML) is a standardized visual language used in software engineering to model, design, and document software systems. It provides a unified notation that allows system elements and their interactions to be represented in a clear and structured manner [1].

UML Diagrams Used in This Project

In this project, the following UML diagrams are used to model the system:

- **Use Case Diagrams:** To represent the functional scope of each module and the interactions between actors and system functionalities.
- **Sequence Diagrams:** To illustrate the dynamic behavior of the system by describing the sequence of interactions between components for key scenarios.
- **Class Diagram:** To represent the static structure of the system, including classes, attributes, methods, and relationships.

2.3.2 Use Case Diagrams

Definition

A use case diagram represents the functional view of a system by illustrating the interactions between external actors and the system. It focuses on the services provided from the user's perspective [2].

Components

A use case diagram includes actors, use cases, a system boundary, and relationships between use cases ($\langle\langle include \rangle\rangle$ and $\langle\langle extend \rangle\rangle$). The $\langle\langle include \rangle\rangle$ relationship indicates that a use case systematically triggers another, while $\langle\langle extend \rangle\rangle$ indicates an optional behavior that may or may not be triggered.

Use Case Diagram — Stock Management

This module involves four internal actors: the Warehouse Keeper, the Stock Manager, the Quality Manager, and the System. The Warehouse Keeper records stock movements and manages storage locations. The Stock Manager consults stock levels

and generates reports. The Quality Manager manages expiry dates, tracks batches, and blocks non-compliant batches. The System automatically triggers low-stock and expiry alerts.

The *Record Stock Movement* use case includes *Track Batch Number*, as every movement must be linked to a batch for traceability. The *Block Non-Compliant Batch* use case also includes *Track Batch Number*, since blocking a batch requires its prior identification.

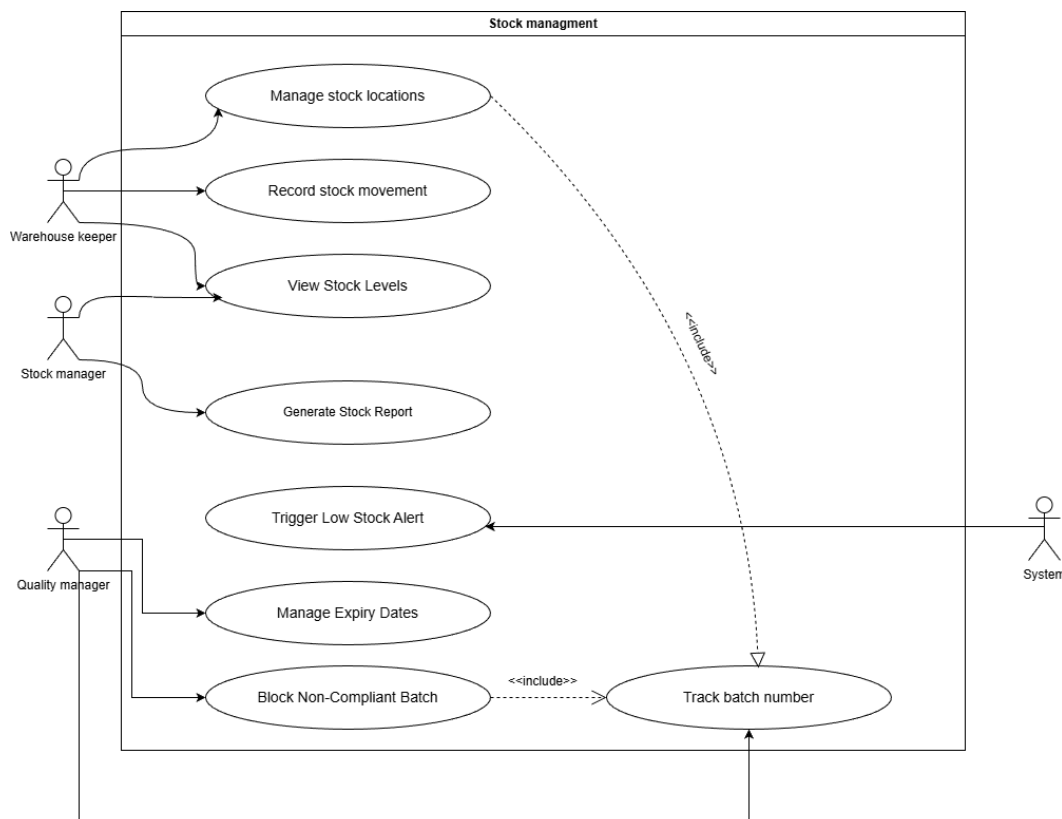


Figure 2.1: Use Case Diagram — Stock Management

Use Case Diagram — Production Management

This module involves four internal actors: the Production Manager, the Operator, the Quality Manager, and the System. The Production Manager creates and plans manufacturing orders, assigns resources, closes orders, and views the production schedule. The Operator tracks progress, records raw material consumption, and views the schedule. The Quality Manager monitors production progress. The System automatically generates batch numbers upon order creation.

The *Create Manufacturing Order* use case includes *Generate Batch Number*, as every order must produce a unique traceable batch. The *Close Manufacturing Order* use case includes *Record Raw Material Consumption*, ensuring all consumed materials are recorded before closure.

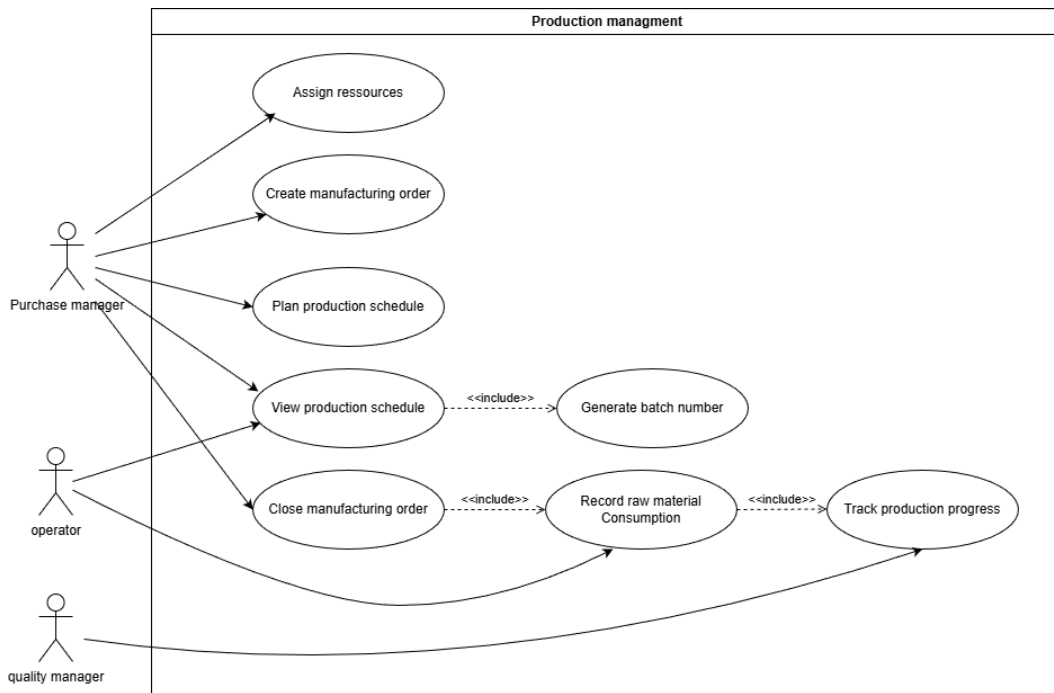


Figure 2.2: Use Case Diagram — Production Management

Use Case Diagram — Purchase Management

This module involves three internal actors: the Purchase Manager, the Stock Manager, and the System. The Purchase Manager creates purchase requests, selects suppliers, generates purchase orders, records invoices, and evaluates suppliers. The Stock Manager receives deliveries and records received batches. The System tracks order statuses automatically.

The *Generate Purchase Order* use case includes *Select Supplier*, as a purchase order cannot be created without an identified supplier. The *Receive Delivery* use case includes *Record Received Batch*, ensuring all received materials are linked to a traceable batch number. The *Evaluate Supplier* use case extends *Track Order Status*, as supplier evaluation is an optional action following order tracking.

Use Case Diagram — Quality Control

This module involves four internal actors: the Quality Manager, the Lab Technician, the Production Manager, and the System. The Quality Manager creates control files, validates or rejects batches, manages non-conformities, triggers batch recalls, and consults quality history. The Lab Technician enters analysis results and generates analysis certificates. The Production Manager consults quality history. The System automatically generates anomaly alerts.

The *Enter Analysis Results* use case includes *Create Control File*, as results can only

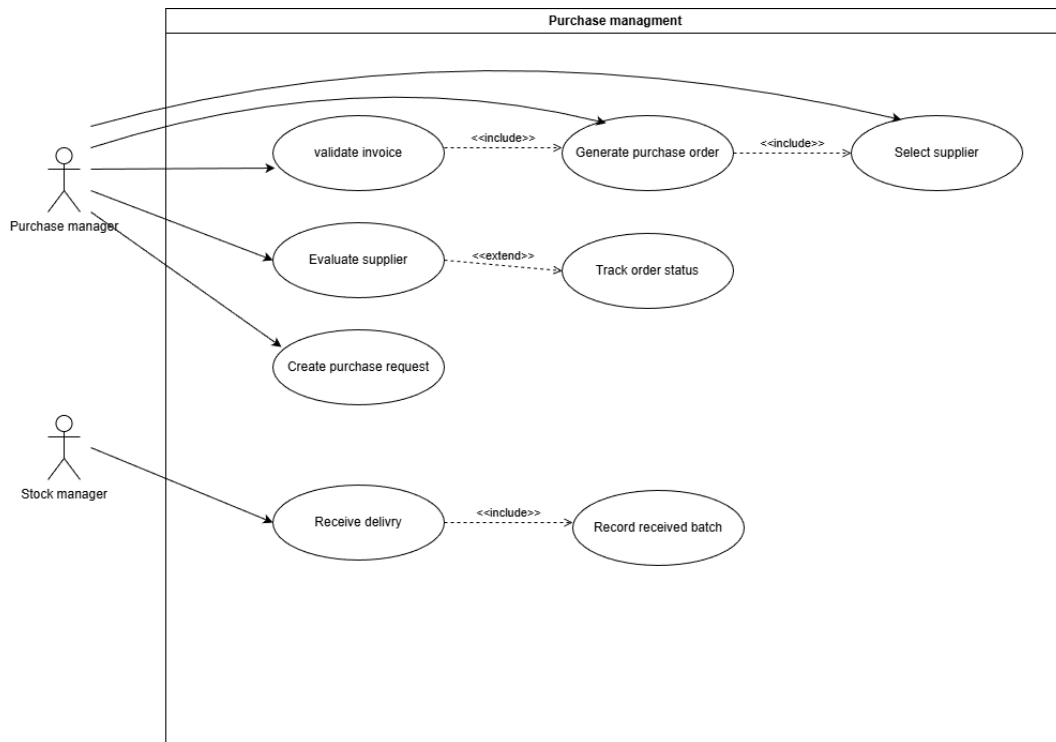


Figure 2.3: Use Case Diagram — Purchase Management

be entered once a control file exists. The *Validate/Reject Batch* use case includes *Enter Analysis Results*, as a decision requires prior analysis. The *Manage Non-Conformities* use case extends *Validate/Reject Batch*, as it is only triggered upon batch rejection. The *Trigger Batch Recall* extends *Manage Non-Conformities*, occurring only in the most serious cases.

Use Case Diagram — Sales Management

This module involves three internal actors (Sales Manager, Stock Manager, System) and one external actor (Client). The Sales Manager creates orders, invoices clients, tracks deliveries, and manages returns. The Stock Manager generates delivery notes and tracks delivered batches. The System automatically checks stock availability. The Client consults the product catalog and tracks their deliveries.

The *Create Sales Order* use case includes *Check Stock Availability*, as availability must always be verified. The *Generate Delivery Note* includes *Track Delivered Batch*, linking every delivery to a traceable batch. The *Invoice Client* includes *Generate Delivery Note*, as invoicing requires a prior delivery note. The *Manage Returns* includes *Track Delivered Batch*, ensuring returned batches are identified for traceability.

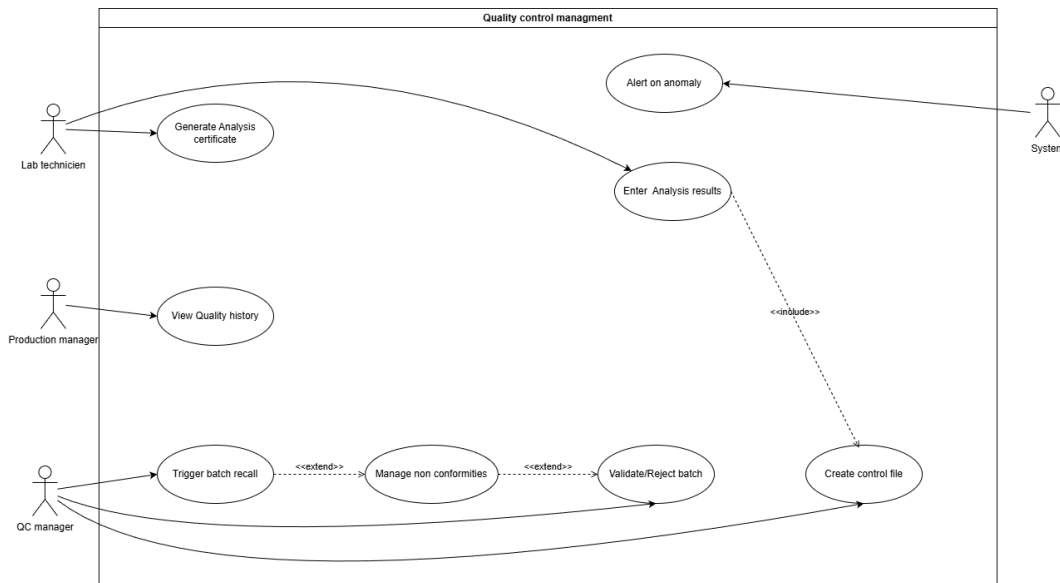


Figure 2.4: Use Case Diagram — Quality Control

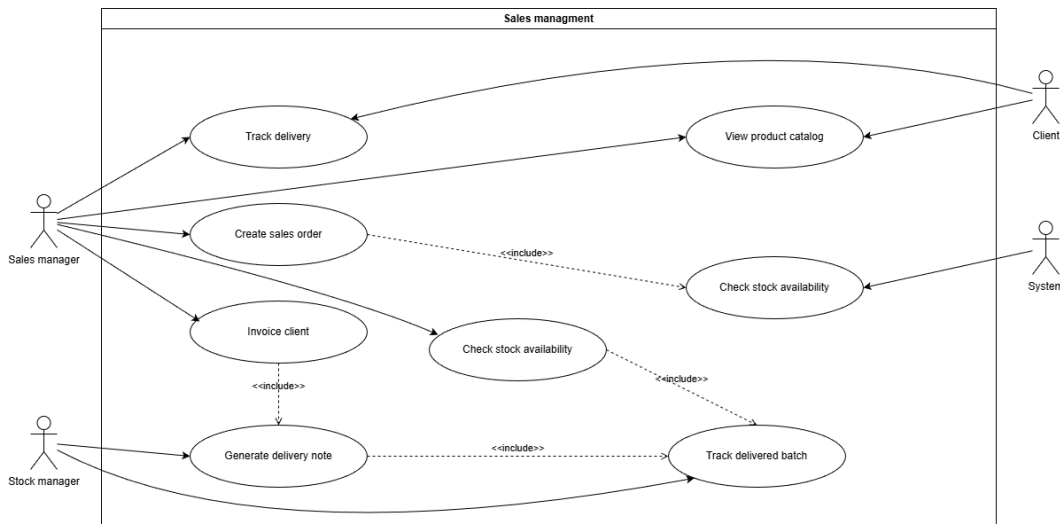


Figure 2.5: Use Case Diagram — Sales Management

2.3.3 Sequence Diagrams

Definition

A sequence diagram describes how system components interact through a sequence of messages over time in a specific scenario [2]. It provides a dynamic view of the system by illustrating the chronological flow of interactions between the user interface, backend services, and the database.

Components

A sequence diagram includes actors, objects, lifelines, messages, activation bars, and conditional fragments (alt, opt).

Sequence Diagrams Explanation

Six sequence diagrams were developed to describe the key dynamic scenarios of the system. Each diagram focuses on a critical use case and illustrates the interactions between the frontend (React.js), the backend (Node.js/Express.js), and the database (PostgreSQL).

- The **Authentication** diagram illustrates how a user submits credentials, how the backend verifies them against the database, and how a JWT token is returned upon successful authentication.
- The **Record Stock Movement** diagram shows how a warehouse keeper records a stock entry or exit, triggering an automatic batch traceability record.
- The **Create Manufacturing Order** diagram describes how a production manager creates a manufacturing order, triggering the automatic generation of a unique batch number by the system.
- The **Validate/Reject Batch** diagram illustrates the quality control decision flow, from result entry by the lab technician to the final validation or rejection decision by the quality manager.
- The **Create Sales Order** diagram shows how a sales manager creates a client order, with automatic stock availability verification and delivery note generation.
- The **Trigger Batch Recall** diagram demonstrates the end-to-end traceability capability of the system, showing how a quality manager can instantly identify all clients affected by a recalled batch.

Authentication Sequence Diagram

The authentication process begins when the user submits their credentials through the login interface. The frontend sends a POST request to the backend authentication endpoint. The backend queries the database to verify the user's existence and validates the submitted password against the stored hash. Upon successful verification, a JWT token is generated and returned to the frontend, which stores it for subsequent authenticated requests. If the credentials are invalid, an error message is returned to the user.

Record Stock Movement Sequence Diagram

The stock movement recording process is initiated by the warehouse keeper through the stock management interface. The user fills in the movement details (product, quantity, type, reason) and submits the form. The frontend sends the data to the backend, which

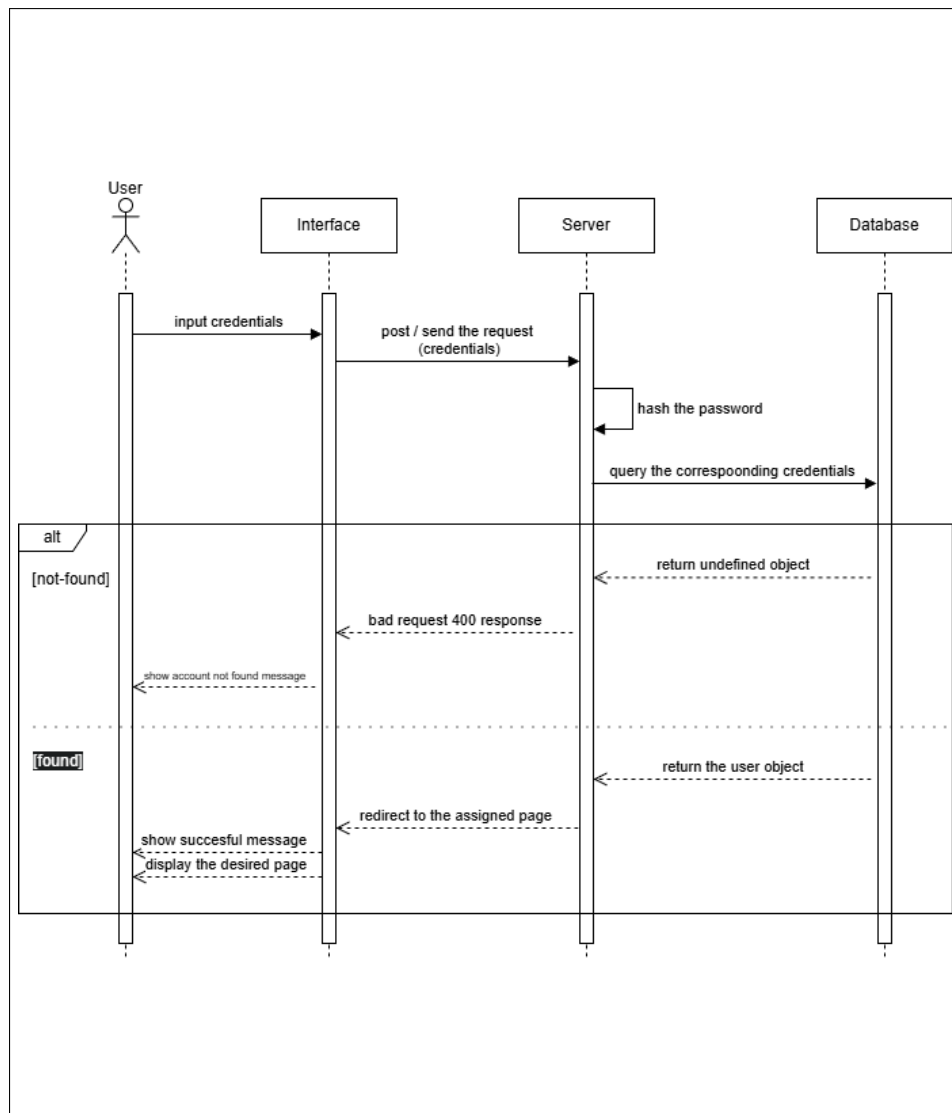
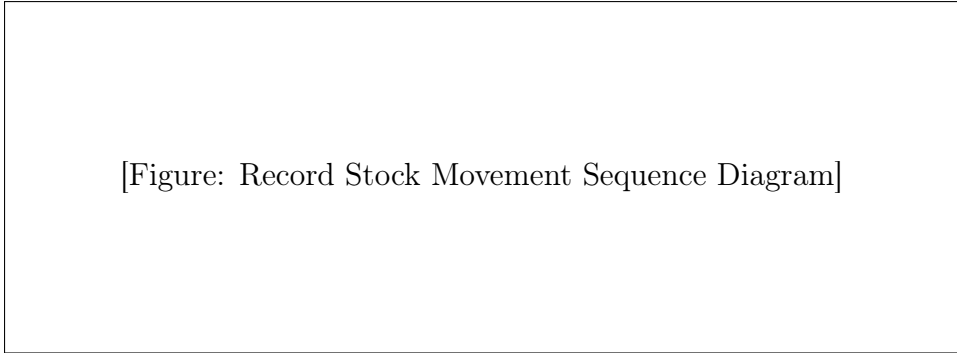


Figure 2.6: Authentication Sequence Diagram

validates the input, records the movement in the database, and automatically links it to the corresponding batch number for traceability. The updated stock level is then returned and displayed to the user.

Create Manufacturing Order Sequence Diagram

The manufacturing order creation process begins when the production manager accesses the production module and fills in the order details (product, quantity, planned date). Upon submission, the frontend sends the data to the backend, which validates the request, automatically generates a unique batch number, creates the manufacturing order record in the database, and returns a confirmation to the user. The generated batch number is immediately visible in the order details.



[Figure: Record Stock Movement Sequence Diagram]

Figure 2.7: Record Stock Movement Sequence Diagram

Validate / Reject Batch Sequence Diagram

The batch validation process begins when the lab technician enters analysis results into the system. The backend stores the results and checks them against predefined quality thresholds. If an anomaly is detected, an automatic alert is sent to the quality manager. The quality manager reviews the results and makes a validation or rejection decision. If the batch is rejected, a non-conformity record is automatically created. The batch status is updated in the database and reflected across all modules.

Create Sales Order Sequence Diagram

The sales order creation process begins when the sales manager selects a client and adds products to the order. Upon submission, the frontend sends the order to the backend, which automatically queries the database to verify stock availability for each product. If stock is sufficient, the order is confirmed, a delivery note is generated, and the corresponding batch numbers are linked to the delivery for traceability. The client and sales manager are notified of the order confirmation.

Trigger Batch Recall Sequence Diagram

The batch recall process is initiated by the quality manager when a serious non-conformity is identified. The quality manager selects the affected batch number and triggers a recall. The backend queries the database to retrieve all sales orders and delivery notes linked to that batch number, identifying every client who received products from this batch. A recall notification is generated for each affected client, and the batch status is updated to recalled across all modules. This scenario demonstrates the core value of the end-to-end traceability mechanism.

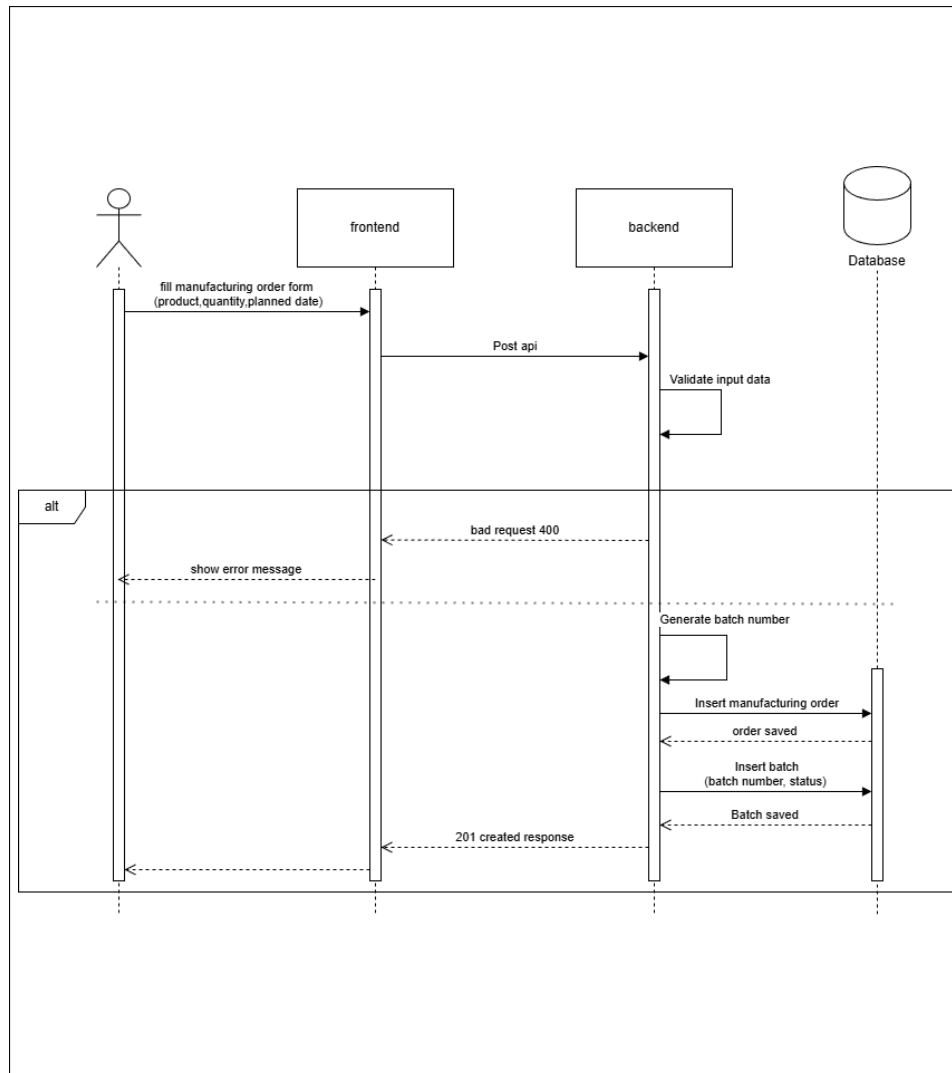


Figure 2.8: Create Manufacturing Order Sequence Diagram

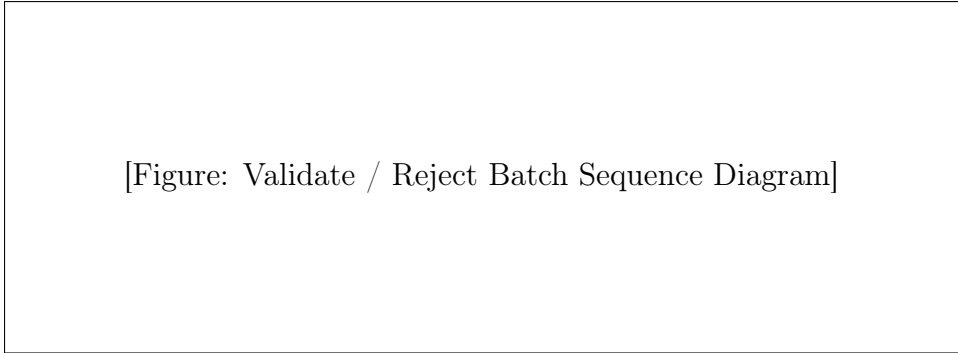
2.3.4 Class Diagram

Definition

A class diagram represents the static structure of a system by describing its classes, attributes, methods, and relationships [2]. It provides a blueprint of the system's data model and serves as the foundation for database design and backend implementation.

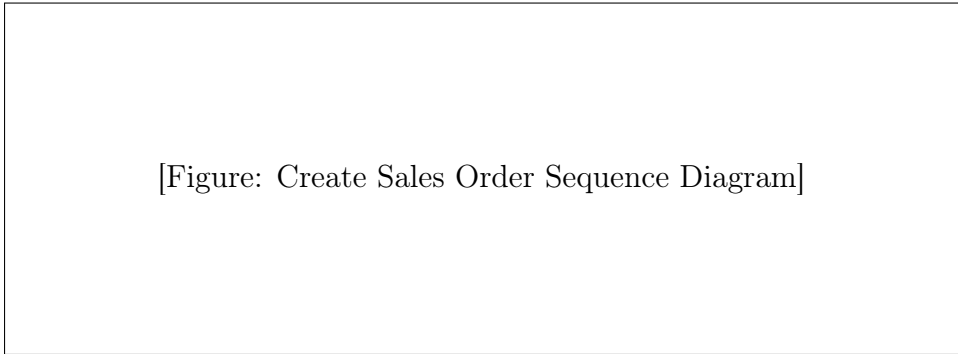
Components

A class diagram includes classes with their attributes and methods, and relationships between classes: associations, aggregations, compositions, and inheritance.



[Figure: Validate / Reject Batch Sequence Diagram]

Figure 2.9: Validate / Reject Batch Sequence Diagram



[Figure: Create Sales Order Sequence Diagram]

Figure 2.10: Create Sales Order Sequence Diagram

Description of the Class Diagram

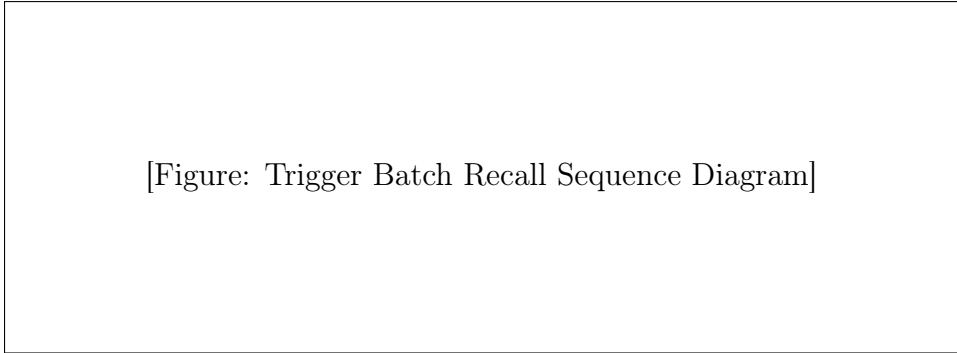
The class diagram of the Inphamedis ERP system is organized around fifteen main classes interconnected through associations that reflect the operational relationships between entities.

The **Batch** class is the central entity of the system, reflecting the end-to-end traceability mechanism. It holds a unique batch number, manufacture date, expiry date, quantity, and status, and is associated with all other operational entities across all modules.

The **Product** class represents pharmaceutical products and raw materials, each linked to multiple batches throughout their lifecycle. The **StockMovement** class records every entry and exit of a batch in the warehouse, while the **StorageLocation** class manages the physical organization of the warehouse.

The **ManufacturingOrder** class represents production orders and generates batches upon creation. The **RawMaterialConsumption** class tracks the materials consumed during each manufacturing order.

The **Supplier** class manages supplier information, while the **PurchaseRequest** and **PurchaseOrder** classes handle the procurement workflow. The **QualityControl** class records control results for each batch, and the **NonConformity** class documents any



[Figure: Trigger Batch Recall Sequence Diagram]

Figure 2.11: Trigger Batch Recall Sequence Diagram

identified quality issues.

The **Client** class stores client information, while the **SalesOrder** and **DeliveryNote** classes manage the sales and distribution workflow. Finally, the **User** class manages system users with their roles and authentication data.

2.3.5 Security Considerations

Authentication and Authorization

The system implements a secure authentication mechanism based on JSON Web Tokens (JWT). Upon successful login, the backend generates a signed token containing the user's identity and role. This token is included in all subsequent API requests and verified by the backend middleware before granting access to any resource.

Role-based access control (RBAC) ensures that each user can only access the functionalities corresponding to their assigned role. For example, a lab technician can enter analysis results but cannot validate batches or trigger recalls, which are restricted to the quality manager.

Data Protection

All user passwords are stored in hashed form using a secure hashing algorithm. Communication between the frontend and backend is conducted over HTTPS to prevent data interception. Sensitive operations such as batch recalls and batch blocking are logged in the system for audit purposes.

Conclusion

This chapter presented the complete analysis and conception of the proposed Inphamedis ERP system. The specification of needs defined the functional and non-functional requirements of the system and identified all actors involved. The conception

phase formalized the system through a comprehensive set of UML models.

The five use case diagrams defined the functional scope of each module and the interactions between users and system functionalities. The six sequence diagrams illustrated the dynamic behavior of the system for key operational scenarios, highlighting the central role of batch traceability across all processes. The class diagram described the static structure of the system and established the foundation for database design and backend implementation.

This design phase provides a solid and coherent foundation for the implementation of the system, which will be presented in the following chapter.

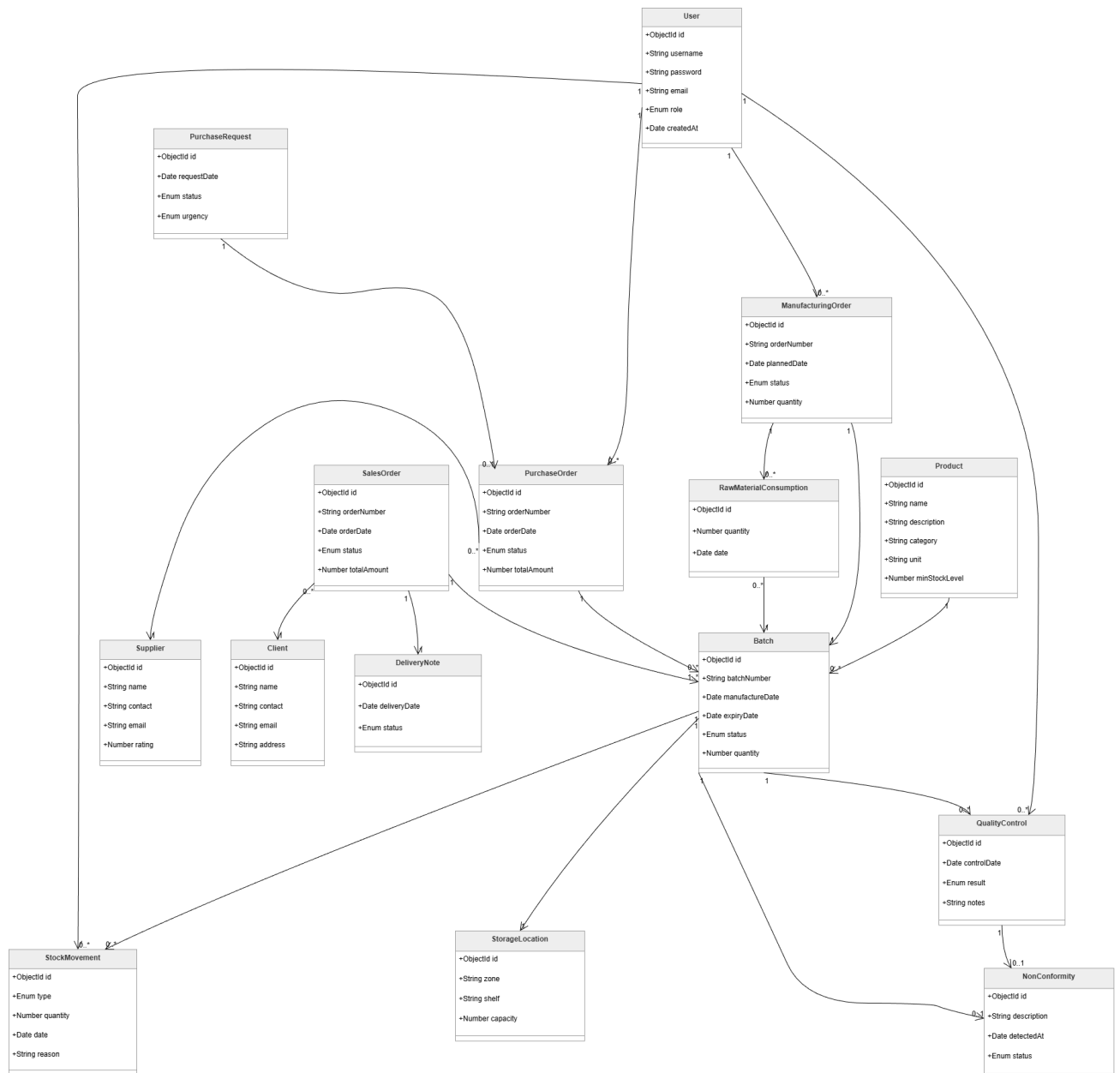


Figure 2.12: Class Diagram of the Inphamedis ERP System

Bibliography

- [1] Object Management Group (OMG), *Unified Modeling Language (UML), Version 2.5.1*, December 2017.
- [2] Sommerville, I., *Software Engineering*, 10th Edition, Pearson, 2015.
- [3] Pressman, R. S., Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill, 2015.